

Milestone Zero

LIME Implementation from Scratch with GPU Acceleration (CUDA / OpenCL)

Haris Khalid - 28557 Aadesh Panjwani - 29303

Hamna Hammad - 29003 Bareerah Shahid - 29385

1 Our Understanding of the Project

The objective of this project is to implement the LIME (Local Interpretable Model-agnostic Explanations) algorithm from scratch and progressively accelerate its computationally intensive components using low-level GPU programming (CUDA or OpenCL). The project emphasizes understanding the internal mechanics of LIME, identifying parallelizable workloads, and evaluating performance trade-offs between CPU and GPU implementations.

LIME is a post-hoc interpretability method designed to explain the prediction of a black-box model for a specific input instance. Instead of interpreting the model globally, LIME approximates the model locally around a selected instance using a simpler, interpretable surrogate model.

For a given input instance, the LIME pipeline consists of the following steps:

1. Generate a large set of perturbed samples around the original input.
2. Run the base model's inference on the perturbed samples.
3. Compute similarity-based weights between the original input and each perturbed sample.
4. Train a local surrogate model using weighted samples.
5. Use the surrogate model parameters as feature attributions.

From a parallel computing perspective, several components of LIME are highly data-parallel:

- **Perturbation Generation:** Each perturbed sample can be generated independently.
- **Model Inference:** Predictions on perturbed samples can be computed in parallel.

- **Similarity and Weight Computation:** Distance and kernel weight calculations are independent per sample.
- **Surrogate Model Training:** Matrix operations and optimization steps are parallelizable.

The project is structured into three milestones. Milestone 1 focuses on GPU acceleration of perturbation, inference, and weight computation without using deep learning frameworks. Milestone 2 completes the pipeline using a high-level framework to validate correctness and interpretability. Milestone 3 replaces the framework-based surrogate training with a custom GPU implementation, resulting in a fully GPU-native LIME system.

Overall, the project requires careful system design, efficient memory management, correctness validation through CPU comparison, and structured performance benchmarking.

2 Division of Tasks for Each Milestone

Our group consists of four members. Responsibilities are divided to ensure modular development, correctness verification, and efficient integration across milestones.

Milestone 1: GPU-based Perturbation, Inference, and Weight Computation

Objective: Implement the computational core of LIME using CUDA/OpenCL without relying on machine learning frameworks.

- **Haris Khalid – GPU Perturbation Module**
 - Design GPU kernels for feature masking and noise injection.
 - Implement random sample generation on GPU.
 - Validate GPU perturbations against CPU reference implementation.
- **Aadesh Panjwani – GPU Base Model Inference**
 - Implement a simple binary classifier (e.g., logistic regression).
 - Develop forward-pass inference kernel.
 - Optimize memory access and parallel execution.
 - Validate GPU inference against CPU results.
- **Hamna Hammad – GPU Similarity and Weight Computation**
 - Implement distance metrics (Euclidean or cosine).
 - Implement kernel-based weighting function.
 - Ensure numerical stability and correctness.

- **Bareerah Shahid – Integration and Benchmarking**

- Handle host-side orchestration and memory transfers.
- Integrate all GPU modules.
- Perform CPU vs GPU performance measurements.
- Document kernel design and data flow.

Milestone 2: Framework-based Surrogate Model Training

Objective: Use a high-level framework to complete the LIME pipeline and validate correctness.

- **Haris Khalid:** Prepare dataset using perturbed samples, predictions, and weights.
- **Aadesh Panjwani:** Implement weighted linear regression or shallow neural network.
- **Hamna Hammad:** Extract surrogate coefficients and generate explanation outputs and visualizations.
- **Bareerah Shahid:** Perform explanation validation using feature ablation and consistency checks.

All members will collaborate in analyzing correctness and comparing framework-assisted results with GPU-accelerated components.

Milestone 3: Fully GPU-based Surrogate Training

Objective: Replace framework-based surrogate training with a custom GPU implementation.

- **Haris Khalid:** Implement weighted linear regression using closed-form solution.
- **Aadesh Panjwani:** Implement required matrix operations on GPU.
- **Hamna Hammad:** Implement iterative optimization (e.g., gradient descent) if required, ensuring numerical stability.
- **Bareerah Shahid:** Integrate end-to-end GPU-native pipeline and conduct final benchmarking:
 - CPU-only implementation
 - Framework-assisted implementation
 - Fully GPU-native implementation

All members will contribute to the final documentation, performance analysis, and system evaluation.